

**1(i) IMPLEMENT BASIC OPERATIONS ON ARRAYS:
INSERTION, DELETION, SEARCHING.**

A) Insertion

Program:

```
#include <stdio.h>

int main() {
    int LA[10] = {0}; // Initialize an array of size 10 with all elements set to 0
    int num_elements;
    printf("Enter the number of elements to insert: ");
    scanf("%d", &num_elements);
    if (num_elements > 10) {
        printf("Error: Number of elements to insert exceeds array size.\n");
        return 1; // Error code to indicate failure
    }
    printf("Enter %d elements to insert:\n", num_elements);
    for (int i = 0; i < num_elements; i++) {
        printf("Enter element %d: ", i + 1);
        scanf("%d", &LA[i]);
    }
    printf("The array elements after insertion :\n");
    for (int i = 0; i < 10; i++) {
        printf("LA[%d] = %d \n", i, LA[i]);
    }
    return 0;
}
```

Output:

Enter the number of elements to insert: 4

Enter 4 elements to insert:

Enter element 1: 3

Enter element 2: 5

Enter element 3: 6

Enter element 4: 7

The array elements after insertion :

LA[0] = 3

LA[1] = 5

LA[2] = 6

LA[3] = 7

LA[4] = 0

LA[5] = 0

LA[6] = 0

LA[7] = 0

LA[8] = 0

LA[9] = 0

B) Deletion

Program:

```
#include <stdio.h>

int main() {
    int n;

    printf("Enter the size of the array: ");
    scanf("%d", &n);

    int LA[n]; // Declaring array of size n

    printf("Enter %d elements for the array:\n", n);
    for(int i = 0; i < n; i++) {
        printf("Enter element %d: ", i + 1);
        scanf("%d", &LA[i]);
    }

    printf("The original array elements are:\n");
    for(int i = 0; i < n; i++)
        printf("LA[%d] = %d \n", i, LA[i]);

    int del_index;

    printf("Enter the index of the element to delete (0 to %d): ", n - 1);
    scanf("%d", &del_index);

    if (del_index < 0 || del_index >= n) {
        printf("Invalid index.\n");
        return 1; // Exiting program with error code
    }

    // Deleting element at del_index
    for(int i = del_index; i < n - 1; i++) {
        LA[i] = LA[i + 1];
    }

    n = n - 1; // Decreasing size of array

    printf("The array elements after deletion:\n");
```

```
for(int i = 0; i < n; i++)  
printf("LA[%d] = %d \n", i, LA[i]);  
return 0;  
}
```

Output:

Enter the size of the array: 3

Enter 3 elements for the array:

Enter element 1: 4

Enter element 2: 5

Enter element 3: 7

The original array elements are:

LA[0] = 4

LA[1] = 5

LA[2] = 7

Enter the index of the element to delete (0 to 2): 1

The array elements after deletion:

LA[0] = 4

LA[1] = 7

C) Searching

Program:

```
#include <stdio.h>

int main() {
    int n;
    printf("Enter the size of the array: ");
    scanf("%d", &n);
    int Arr[n];
    printf("Enter %d elements for the array:\n", n);
    for (int i = 0; i < n; i++) {
        printf("Enter element %d: ", i + 1);
        scanf("%d", &Arr[i]);
    }
    int item;
    printf("Enter the element to search: ");
    scanf("%d", &item);
    int j = 0;
    while (j < n) {
        if (Arr[j] == item) {
            break;
        }
        j++;
    }
    if (j < n) {
        printf("Found element %d at position %d\n", item, j + 1);
    } else {
        printf("Element %d not found in the array\n", item);
    }
    return 0;
}
```

}

Output:

Enter the size of the array: 4

Enter 4 elements for the array:

Enter element 1: 3

Enter element 2: 5

Enter element 3: 7

Enter element 4: 8

Enter the element to search: 5

Found element 5 at position 2

**1 (ii) CREATE A PROGRAM TO FIND THE MAXIMUM AND
MINIMUM ELEMENTS IN AN ARRAY.**

Program:

```
#include <limits.h>
#include <stdio.h>
// Function to find the minimum and
// maximum element of the array
void findMinimumMaximum(int arr[], int N)
{
    int i;
    // variable to store the minimum
    // and maximum element
    int minE = INT_MAX, maxE = INT_MIN;
    // Traverse the given array
    for (i = 0; i < N; i++) {
        // If current element is smaller
        // than minE then update it
        if (arr[i] < minE) {
            minE = arr[i];
        }
        // If current element is greater
        // than maxE then update it
        if (arr[i] > maxE) {
            maxE = arr[i];
        }
    }
    // Print the minimum and maximum element
    printf("The minimum element is %d\n", minE);
    printf("The maximum element is %d\n", maxE);
}
```

```
return;
}
// Driver Code
int main()
{
    int N;
    printf("Enter the number of elements in the array: ");
    scanf("%d", &N);
    // Given array
    int arr[N];
    printf("Enter the elements of the array:\n");
    for (int i = 0; i < N; i++) {
        scanf("%d", &arr[i]);
    }
    // Function call
    findMinimumMaximum(arr, N);
    return 0;
}
```

Output:

Enter the number of elements in the array: 3

Enter the elements of the array:

7

9

6

The minimum element is 6

The maximum element is 9

1 (iii) WRITE A PROGRAM TO REVERSE AN ARRAY.

Program:

```
#include<stdio.h>

int main()
{
    int i, n, temp;
    printf("Enter the size of the array: ");
    scanf("%d", &n);
    int arr[n];
    printf("Enter the elements: ");
    for(i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }
    int end = n - 1;
    for(i = 0; i < n/2; i++)
    {
        temp = arr[i];
        arr[i] = arr[end];
        arr[end] = temp;
        end--;
    }
    printf("The reversed array: ");
    for(i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }
}
```

Output:

Enter the size of the array: 5

Enter the elements: 2 5 7 8 9

The reversed array: 9 8 7 5 2

2(i)IMPLEMENT A SINGLY LINKED LIST AND PERFORM INSERTION AND DELETION OPERATIONS.

Program:

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node* next;
};
void insert(struct Node** head, int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;
    if (*head == NULL) *head = newNode;
    else {
        struct Node* last = *head;
        while (last->next != NULL) last = last->next;
        last->next = newNode;
    }
}
void deleteNode(struct Node** head, int key) {
    struct Node* temp = *head, *prev = NULL;
    if (temp != NULL && temp->data == key) {
        *head = temp->next;
        free(temp);
        return;
    }
    while (temp != NULL && temp->data != key) {
        prev = temp;
```

```

temp = temp->next;
}
if (temp == NULL) return;
prev->next = temp->next;
free(temp);
}
void printList(struct Node* head) {
while (head != NULL) {
printf("%d -> ", head->data);
head = head->next;
}
printf("NULL\n");
}
int main() {
struct Node* head = NULL;
int n, data, i;
printf("Enter the number of elements: ");
scanf("%d", &n);
printf("Enter the elements:\n");
for (i = 0; i < n; i++) {
scanf("%d", &data);
insert(&head, data);
}
printf("Linked list: ");
printList(head);
printf("Enter the element to delete: ");
scanf("%d", &data);
deleteNode(&head, data);
printf("Linked list after deletion: ");
printList(head);

```

```
return 0;  
}
```

Output:

Enter the number of elements: 5

Enter the elements:

4

8

9

7

3

Linked list: 4 -> 8 -> 9 -> 7 -> 3 -> NULL

Enter the element to delete: 7

Linked list after deletion: 4 -> 8 -> 9 -> 3 -> NULL

**2(ii) DEVELOP A PROGRAM TO REVERSE A LINKED LIST
ITERATIVELY AND RECURSIVELY.**

Program:

```
#include <stdio.h>
#include <stdlib.h>
typedef struct Node {
    int data;
    struct Node* next;
} Node;

void insert(Node** head, int data) {
    Node* newNode = malloc(sizeof(Node));
    newNode->data = data;
    newNode->next = *head;
    *head = newNode;
}

void printList(Node* node) {
    for (; node != NULL; node = node->next)
        printf("%d -> ", node->data);
    printf("NULL\n");
}

void reverseIterative(Node** head) {
    Node *prev = NULL, *current = *head, *next = NULL;
    while (current) {
        next = current->next;
        current->next = prev;
        prev = current;
        current = next;
    }
    *head = prev;
}
```

```

}
void reverseRecursive(Node** head) {
    if (!*head || !(*head)->next) return;
    Node *first = *head, *rest = first->next;
    reverseRecursive(&rest);
    first->next->next = first;
    first->next = NULL;
    *head = rest;
}
int main() {
    Node* head = NULL;
    int n, data;
    printf("Enter the number of elements: ");
    scanf("%d", &n);
    printf("Enter the elements:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &data);
        insert(&head, data);
    }
    printf("Original linked list: ");
    printList(head);
    reverseIterative(&head);
    printf("Reversed linked list (iterative): ");
    printList(head);
    reverseRecursive(&head);
    printf("Reversed linked list (recursive): ");
    printList(head);
    return 0;
}

```

Output:

Enter the number of elements: 6

Enter the elements:

1

5

3

7

9

2

Original linked list: 2 -> 9 -> 7 -> 3 -> 5 -> 1 -> NULL

Reversed linked list (iterative): 1 -> 5 -> 3 -> 7 -> 9 -> 2 -> NULL

Reversed linked list (recursive): 2 -> 9 -> 7 -> 3 -> 5 -> 1 -> NULL

2(iii) SOLVE PROBLEMS INVOLVING LINKED LIST TRAVERSAL AND MANIPULATION.

Program:

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node* next;
};
struct Node* createNode(int data) {
    struct Node* newNode = malloc(sizeof(struct Node));
    if (!newNode) exit(1);
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}
void insertAtBeginning(struct Node** head, int data) {
    struct Node* newNode = createNode(data);
    newNode->next = *head;
    *head = newNode;
}
void insertAtEnd(struct Node** head, int data) {
    struct Node* newNode = createNode(data);
    if (!*head) *head = newNode;
    else {
        struct Node* lastNode = *head;
        while (lastNode->next) lastNode = lastNode->next;
        lastNode->next = newNode;
    }
}
```

```

}
void deleteNode(struct Node** head, int key) {
    if (!*head) return;
    struct Node *temp = *head, *prev = NULL;
    if (temp && temp->data == key) {
        *head = temp->next;
        free(temp);
        return;
    }
    while (temp && temp->data != key) {
        prev = temp;
        temp = temp->next;
    }
    if (!temp) return;
    prev->next = temp->next;
    free(temp);
}

void printLinkedList(struct Node* head) {
    while (head) {
        printf("%d ", head->data);
        head = head->next;
    }
    printf("\n");
}

int main() {
    struct Node* head = NULL;
    int n, value, deleteValue;
    printf("Enter the number of elements to insert at the beginning: ");
    scanf("%d", &n);
    printf("Enter %d elements:\n", n);

```

```
for (int i = 0; i < n; i++) {
scanf("%d", &value);
insertAtBeginning(&head, value);
}

printf("Linked list after inserting nodes at the beginning: ");
printLinkedList(head);

printf("Enter the number of elements to insert at the end: ");
scanf("%d", &n);
printf("Enter %d elements:\n", n);
for (int i = 0; i < n; i++) {
scanf("%d", &value);
insertAtEnd(&head, value);
}

printf("Linked list after inserting nodes at the end: ");
printLinkedList(head);

printf("Enter the value to delete: ");
scanf("%d", &deleteValue);
deleteNode(&head, deleteValue);
printf("Linked list after deleting node with value %d: ", deleteValue);
printLinkedList(head);
return 0;
}
```

Output:

Enter the number of elements to insert at the beginning: 3

Enter 3 elements:

4

5

6

Linked list after inserting nodes at the beginning: 6 5 4

Enter the number of elements to insert at the end: 4

Enter 4 elements:

8

9

2

1

Linked list after inserting nodes at the end: 6 5 4 8 9 2 1

Enter the value to delete: 8

Linked list after deleting node with value 8: 6 5 4 9 2 1

3 (i) CREATE A PROGRAM TO DETECT AND REMOVE DUPLICATES FROM A LINKED LIST.

Program:

```
#include <stdio.h>
#include <stdlib.h>
struct node{
int data;
struct node *next;
};
struct node *head, *tail = NULL;
void addNode(int data) {
struct node *newNode = (struct node*)malloc(sizeof(struct node));
newNode->data = data;
newNode->next = NULL;
if(head == NULL) {
head = newNode;
tail = newNode;
} else {
tail->next = newNode;
tail = newNode;
}
}
void removeDuplicate() {
struct node *current = head, *index = NULL, *temp = NULL;
if(head == NULL) {
return;
} else {
while(current != NULL){
temp = current;
```

```

index = current->next;
while(index != NULL) {
if(current->data == index->data) {
temp->next = index->next;
} else {
temp = index;
}
index = index->next;
}
current = current->next;
}
}
}

void display() {
struct node *current = head;
if(head == NULL) {
printf("List is empty \n");
return;
}
while(current != NULL) {
printf("%d ", current->data);
current = current->next;
}
printf("\n");
}

int main() {
int num_elements, element;
printf("Enter the number of elements: ");
scanf("%d", &num_elements);
printf("Enter the elements: ");

```

```
for (int i = 0; i < num_elements; i++) {  
    scanf("%d", &element);  
    addNode(element);  
}  
printf("Original list: \n");  
display();  
removeDuplicate();  
printf("List after removing duplicates: \n");  
display();  
return 0;  
}
```

Output:

Enter the number of elements: 5

Enter the elements:

1

2

3

5

1

Original list:

1 2 3 5 1

List after removing duplicates:

1 2 3 5

3(ii) IMPLEMENT A LINKED LIST TO REPRESENT POLYNOMIALS AND PERFORM ADDITION.

Program:

```
#include <stdio.h>
#include <stdlib.h>
struct Term {
    int coefficient, exponent;
    struct Term *next;
};

struct Term* createTerm(int coefficient, int exponent) {
    struct Term* newTerm = (struct Term*)malloc(sizeof(struct Term));
    newTerm->coefficient = coefficient;
    newTerm->exponent = exponent;
    newTerm->next = NULL;
    return newTerm;
}

void insertTerm(struct Term** poly, int coefficient, int exponent) {
    struct Term* newTerm = createTerm(coefficient, exponent);
    if (!*poly) *poly = newTerm;
    else {
        struct Term* temp = *poly;
        while (temp->next) temp = temp->next;
        temp->next = newTerm;
    }
}

void displayPolynomial(struct Term* poly) {
    while (poly) {
        printf("%dx^%d%s", poly->coefficient, poly->exponent, poly->next ? " + " :
        "");
    }
}
```

```

poly = poly->next;
}
printf("\n");
}

struct Term* addPolynomials(struct Term* poly1, struct Term* poly2) {
struct Term *result = NULL, *temp;
while (poly1 || poly2) {
if (poly1 && (!poly2 || poly1->exponent > poly2->exponent))
temp = createTerm(poly1->coefficient, poly1->exponent), poly1 = poly1-
>next;
else if (poly2 && (!poly1 || poly1->exponent < poly2->exponent))
temp = createTerm(poly2->coefficient, poly2->exponent), poly2 = poly2-
>next;
else temp = createTerm(poly1->coefficient + poly2->coefficient, poly1->
exponent), poly1 =
poly1->next, poly2 = poly2->next;
insertTerm(&result, temp->coefficient, temp->exponent);
}
return result;
}

int main() {
struct Term *poly1 = NULL, *poly2 = NULL, *result = NULL;
int coefficient, exponent, num_terms;
printf("Enter the number of terms for polynomial 1: ");
scanf("%d", &num_terms);
printf("Enter coefficients and exponents for polynomial 1:\n");
while (num_terms--) scanf("%d %d", &coefficient, &exponent),
insertTerm(&poly1,
coefficient, exponent);
printf("Enter the number of terms for polynomial 2: ");

```

```
scanf("%d", &num_terms);
printf("Enter coefficients and exponents for polynomial 2:\n");
while (num_terms--) scanf("%d %d", &coefficient, &exponent),
insertTerm(&poly2,
coefficient, exponent);
printf("Polynomial 1: ");
displayPolynomial(poly1);
printf("Polynomial 2: ");
displayPolynomial(poly2);
result = addPolynomials(poly1, poly2);
printf("Result of addition: ");
displayPolynomial(result);
while (poly1) poly1 = poly1->next, free(poly1);
while (poly2) poly2 = poly2->next, free(poly2);
while (result) result = result->next, free(result);
return 0;
}
```

Output:

Enter the number of terms for polynomial 1: 2

Enter coefficients and exponents for polynomial 1:

4

5

2

3

Enter the number of terms for polynomial 2: 2

Enter coefficients and exponents for polynomial 2:

4

8

6

2

Polynomial 1: $4x^5 + 2x^3$

Polynomial 2: $4x^8 + 6x^2$

Result of addition: $4x^8 + 4x^5 + 2x^3 + 6x^2$

3(iii) IMPLEMENT A DOUBLE-ENDED QUEUE (DEQUE) WITH ESSENTIAL OPERATIONS.

Program:

```
#include <stdio.h>
#include <stdlib.h>
#define MAX_SIZE 100
struct Deque {
    int arr[MAX_SIZE];
    int front, rear;
};
void initDeque(struct Deque *deque) {
    deque->front = deque->rear = -1;
}
int isEmpty(struct Deque *deque) {
    return (deque->front == -1);
}
int isFull(struct Deque *deque) {
    return ((deque->rear + 1) % MAX_SIZE == deque->front);
}
void addFront(struct Deque *deque, int value) {
    if (!isFull(deque)) {
        if (isEmpty(deque)) deque->front = deque->rear = 0;
        else deque->front = (deque->front - 1 + MAX_SIZE) % MAX_SIZE;
        deque->arr[deque->front] = value;
    }
}
void addRear(struct Deque *deque, int value) {
    if (!isFull(deque)) {
        if (isEmpty(deque)) deque->front = deque->rear = 0;
```

```

else deque->rear = (deque->rear + 1) % MAX_SIZE;
deque->arr[deque->rear] = value;
}
}

void removeFront(struct Deque *deque) {
if (!isEmpty(deque)) {
if (deque->front == deque->rear) deque->front = deque->rear = -1;
else deque->front = (deque->front + 1) % MAX_SIZE;
}
}

void removeRear(struct Deque *deque) {
if (!isEmpty(deque)) {
if (deque->front == deque->rear) deque->front = deque->rear = -1;
else deque->rear = (deque->rear - 1 + MAX_SIZE) % MAX_SIZE;
}
}

int getFront(struct Deque *deque) {
return isEmpty(deque) ? -1 : deque->arr[deque->front];
}

int getRear(struct Deque *deque) {
return isEmpty(deque) ? -1 : deque->arr[deque->rear];
}

void displayDeque(struct Deque *deque) {
if (isEmpty(deque)) return;
printf("Deque: ");
for (int i = deque->front; i != (deque->rear + 1) % MAX_SIZE; i = (i + 1) %
MAX_SIZE)
printf("%d ", deque->arr[i]);
printf("\n");
}

```

```

}
int main() {
    struct Deque deque;
    initDeque(&deque);
    int value;
    printf("Enter the number of elements to add: ");
    int num;
    scanf("%d", &num);
    printf("Enter the elements to add at front:\n");
    for (int i = 0; i < num; ++i) {
        scanf("%d", &value);
        addFront(&deque, value);
    }
    printf("Enter the number of elements to add: ");
    scanf("%d", &num);
    printf("Enter the elements to add at rear:\n");
    for (int i = 0; i < num; ++i) {
        scanf("%d", &value);
        addRear(&deque, value);
    }
    displayDeque(&deque);
    removeFront(&deque);
    removeRear(&deque);
    displayDeque(&deque);
    printf("Front element: %d\n", getFront(&deque));
    printf("Rear element: %d\n", getRear(&deque));
    return 0;
}

```

Output:

Enter the number of elements to add: 5

Enter the elements to add at front:

1

2

3

4

5

Enter the number of elements to add: 3

Enter the elements to add at rear:

6

7

8

Deque: 5 4 3 2 1 6 7 8

Deque: 4 3 2 1 6 7

Front element: 4

Rear element: 7

**4(i) IMPLEMENT A DOUBLY LINKED LIST AND PERFORM
VARIOUS OPERATIONS TO UNDERSTAND ITS PROPERTIES
AND APPLICATIONS.**

Program:

```
#include <stdio.h>
#include <stdlib.h>
typedef struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
} Node;

Node* createNode(int data) {
    Node* newNode = malloc(sizeof(Node));
    newNode->data = data;
    return newNode;}

void insertAtEnd(Node** head, int data) {
    Node* newNode = createNode(data), *temp = *head;
    if (!*head) *head = newNode;
    else {
        while (temp->next) temp = temp->next;
        temp->next = newNode;
        newNode->prev = temp;
    }
}

void deleteNode(Node** head, int data) {
    Node* current = *head;
    while (current && current->data != data) current = current->next;
    if (!current) printf("Not found\n");
    else {
        if (current->prev) current->prev->next = current->next;
```

```

else *head = current->next;
if (current->next) current->next->prev = current->prev;
free(current);
printf("Deleted\n");
}}
void displayList(Node* head) {
while (head) printf("%d ", head->data), head = head->next;
printf("\n");}
int main() {
Node* head = NULL;
int choice, data;
while (1) {
printf("1. Insert\n2. Delete\n3. Display\n4. Exit\nEnter your choice: ");
scanf("%d", &choice);
switch (choice) {
case 1:
printf("Enter data: ");
scanf("%d", &data);
insertAtEnd(&head, data);
break;
case 2:
printf("Enter data to delete: ");
scanf("%d", &data);
deleteNode(&head, data);
break;
case 3:
displayList(head);
break;
case 4:
exit(0);
}
}
}

```

```
default:  
printf("Invalid choice\n");  
}}  
return 0;}
```

Output:

```
1. Insert  
2. Delete  
3. Display  
4. Exit  
Enter your choice: 1  
Enter data: 4  
1. Insert  
2. Delete  
3. Display  
4. Exit  
Enter your choice: 1  
Enter data: 6  
1. Insert  
2. Delete  
3. Display  
4. Exit  
Enter your choice: 1  
Enter data: 8  
1. Insert  
2. Delete  
3. Display  
4. Exit  
Enter your choice: 3  
4 6 8
```

**4(ii) IMPLEMENT A CIRCULAR LINKED LIST AND
PERFORM INSERTION, DELETION, AND TRAVERSAL.**

Program:

```
#include <stdio.h>
#include <stdlib.h>
typedef struct Node {
    int data;
    struct Node* next;
} Node;

Node* createNode(int data) {
    Node* newNode = malloc(sizeof(Node));
    if (!newNode) {
        perror("Memory allocation failed");
        exit(EXIT_FAILURE);
    }
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}

void insert(Node** head, int data) {
    Node* newNode = createNode(data);
    if (!*head) {
        *head = newNode;
        newNode->next = *head;
    } else {
        newNode->next = (*head)->next;
        (*head)->next = newNode;
    }
}
```

```

void delete(Node** head, int data) {
    if (!*head) {
        printf("List is empty!\n");
        return;
    }
    Node* temp = *head;
    Node* prev = NULL;
    do {
        if (temp->data == data) {
            if (temp == *head) {
                *head = temp->next;
            }
            prev->next = temp->next;
            free(temp);
            return;
        }
        prev = temp;
        temp = temp->next;
    } while (temp != *head);
    printf("Element not found in the list!\n");
}

void traverse(Node* head) {
    if (!head) {
        printf("List is empty!\n");
        return;
    }
    Node* temp = head;
    do {
        printf("%d ", temp->data);

```

```

temp = temp->next;
} while (temp != head);
printf("\n");
}

int main() {
Node* head = NULL;
int choice, data;
while (1) {
printf("\n1. Insert\n2. Delete\n3. Traverse\n4. Exit\n");
printf("Enter your choice: ");
scanf("%d", &choice);
switch (choice) {
case 1:
printf("Enter data to insert: ");
scanf("%d", &data);
insert(&head, data);
break;
case 2:
printf("Enter data to delete: ");
scanf("%d", &data);
delete(&head, data);
break;
case 3:
printf("Circular Linked List: ");
traverse(head);
break;
case 4:
exit(EXIT_SUCCESS);
default:
printf("Invalid choice! Please try again.\n");

```

```
}  
}  
return 0;  
}
```

Output:

1. Insert

2. Delete

3. Traverse

4. Exit

Enter your choice: 1

Enter data to insert: 2

1. Insert

2. Delete

3. Traverse

4. Exit

Enter your choice: 1

Enter data to insert: 8

1. Insert

2. Delete

3. Traverse

4. Exit

Enter your choice: 1

Enter data to insert: 9

1. Insert

2. Delete

3. Traverse

4. Exit

Enter your choice: 3

Circular Linked List: 2 9 8

5(i) IMPLEMENT A STACK USING ARRAYS AND LINKED LISTS.

Program:

```
#include <stdio.h>
#include <stdlib.h>
#define MAX_SIZE 100
struct Node {
    int data;
    struct Node* next;
};
struct ArrayStack {
    int top;
    int array[MAX_SIZE];
};
void pushLinkedList(struct Node** root, int data) {
    struct Node* newNode = malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = *root;
    *root = newNode;
    printf("%d pushed to linked list stack\n", data);
}
int popLinkedList(struct Node** root) {
    if (!*root) {
        printf("Linked list stack underflow\n");
        exit(EXIT_FAILURE);
    }
    int popped = (*root)->data;
    struct Node* temp = *root;
    *root = (*root)->next;
```

```

free(temp);
return popped;
}

void pushArray(struct ArrayStack* stack, int data) {
if (stack->top >= MAX_SIZE - 1) {
printf("Array stack overflow\n");
exit(EXIT_FAILURE);
}
stack->array[++stack->top] = data;
printf("%d pushed to array stack\n", data);
}

int popArray(struct ArrayStack* stack) {
if (stack->top == -1) {
printf("Array stack underflow\n");
exit(EXIT_FAILURE);
}
return stack->array[stack->top--];
}

void printLinkedList(struct Node* root) {
printf("Linked List Stack: ");
while (root) {
printf("%d ", root->data);
root = root->next;
}
printf("\n");
}

void printArray(struct ArrayStack* stack) {
printf("Array Stack: ");
for (int i = 0; i <= stack->top; i++)

```

```

printf("%d ", stack->array[i]);
printf("\n");
}

int main() {
int choice, data;
struct Node* root = NULL;
struct ArrayStack stack = { .top = -1 };
while (1) {
printf("\nStack Operations\n");
printf("1. Push (LL)\n");
printf("2. Pop (LL)\n");
printf("3. Push (Array)\n");
printf("4. Pop (Array)\n");
printf("5. View LL Stack\n");
printf("6. View Array Stack\n");
printf("7. Exit\n");
printf("Enter choice: ");
scanf("%d", &choice);
switch (choice) {
case 1:
printf("Enter data to push (LL): ");
scanf("%d", &data);
pushLinkedList(&root, data);
break;
case 2:
printf("%d popped from LL stack\n", popLinkedList(&root));
break;
case 3:
printf("Enter data to push (Array): ");

```

```
scanf("%d", &data);
pushArray(&stack, data);
break;
case 4:
printf("%d popped from array stack\n", popArray(&stack));
break;
case 5:
printLinkedList(root);
break;
case 6:
printArray(&stack);
break;
case 7:
printf("Exiting...\n");
exit(0);
default:
printf("Invalid choice\n");
}
}
return 0;
}
```

Output:

Stack Operations

1. Push (LL)
2. Pop (LL)
3. Push (Array)
4. Pop (Array)
5. View LL Stack
6. View Array Stack
7. Exit

Enter choice: 1

Enter data to push (LL): 6

6 pushed to linked list stack

Stack Operations

1. Push (LL)
2. Pop (LL)
3. Push (Array)
4. Pop (Array)
5. View LL Stack
6. View Array Stack
7. Exit

Enter choice: 1

Enter data to push (LL): 9

9 pushed to linked list stack

Stack Operations

1. Push (LL)
2. Pop (LL)
3. Push (Array)
4. Pop (Array)
5. View LL Stack

6. View Array Stack

7. Exit

Enter choice: 5

Linked List Stack: 9 6

Stack Operations

1. Push (LL)

2. Pop (LL)

3. Push (Array)

4. Pop (Array)

5. View LL Stack

6. View Array Stack

7. Exit

Enter choice: 7

Exiting...

**5(ii) WRITE A PROGRAM TO EVALUATE A POSTFIX
EXPRESSION USING A STACK.**

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#define MAX_SIZE 100
// Structure for Stack
struct Stack {
    int top;
    int array[MAX_SIZE];
};
// Function to create a stack
struct Stack* createStack() {
    struct Stack* stack = (struct Stack*)malloc(sizeof(struct Stack));
    if (!stack) {
        printf("Memory allocation error\n");
        exit(EXIT_FAILURE);
    }
    stack->top = -1;
    return stack;
}
// Function to push an element onto the stack
void push(struct Stack* stack, int data) {
    if (stack->top >= MAX_SIZE - 1) {
        printf("Stack overflow\n");
        exit(EXIT_FAILURE);
    }
    stack->array[++stack->top] = data;
```

```

}

// Function to pop an element from the stack
int pop(struct Stack* stack) {
    if (stack->top == -1) {
        printf("Stack underflow\n");
        exit(EXIT_FAILURE);
    }
    return stack->array[stack->top--];
}

// Function to evaluate postfix expression
int evaluatePostfix(char* exp) {
    struct Stack* stack = createStack();
    int i, operand1, operand2;
    for (i = 0; exp[i]; ++i) {
        if (isdigit(exp[i])) {
            push(stack, exp[i] - '0');
        } else {
            operand2 = pop(stack);
            operand1 = pop(stack);
            switch (exp[i]) {
                case '+':
                    push(stack, operand1 + operand2);
                    break;
                case '-':
                    push(stack, operand1 - operand2);
                    break;
                case '*':
                    push(stack, operand1 * operand2);
                    break;
                case '/':

```

```
push(stack, operand1 / operand2);
break;
}
}
}
return pop(stack);
}

int main() {
char exp[MAX_SIZE];
printf("Enter a postfix expression: ");
scanf("%s", exp);
printf("Result: %d\n", evaluatePostfix(exp));
return 0;
}
```

Output:

```
Enter a postfix expression: 575*+
Result: 40
```

5(iii) IMPLEMENT A PROGRAM TO CHECK FOR BALANCED PARENTHESES USING A STACK.

Program:

```
#include <stdio.h>
#include <stdlib.h>
#define MAX_SIZE 100
// Structure for Stack
struct Stack {
    int top;
    char array[MAX_SIZE];
};
// Function to create a stack
struct Stack* createStack() {
    struct Stack* stack = (struct Stack*)malloc(sizeof(struct Stack));
    if (!stack) {
        printf("Memory allocation error\n");
        exit(EXIT_FAILURE);
    }
    stack->top = -1;
    return stack;
}
// Function to push an element onto the stack
void push(struct Stack* stack, char data) {
    if (stack->top >= MAX_SIZE - 1) {
        printf("Stack overflow\n");
        exit(EXIT_FAILURE);
    }
    stack->array[++stack->top] = data;
}
```

```

// Function to pop an element from the stack
char pop(struct Stack* stack) {
    if (stack->top == -1) {
        printf("Stack underflow\n");
        exit(EXIT_FAILURE);
    }
    return stack->array[stack->top--];
}

// Function to check if character is an opening bracket
int isOpeningBracket(char c) {
    return (c == '(' || c == '{' || c == '[');
}

// Function to check if character is a closing bracket
int isClosingBracket(char c) {
    return (c == ')' || c == '}' || c == ']');
}

// Function to check if brackets are balanced
int areBracketsBalanced(char* exp) {
    struct Stack* stack = createStack();
    for (int i = 0; exp[i]; i++) {
        if (isOpeningBracket(exp[i])) {
            push(stack, exp[i]);
        } else if (isClosingBracket(exp[i])) {
            if (stack->top == -1) {
                return 0;
            } else if ((exp[i] == ')' && stack->array[stack->top] != '(') ||
                (exp[i] == '}' && stack->array[stack->top] != '{') ||
                (exp[i] == ']' && stack->array[stack->top] != '[')) {
                return 0;
            } else {

```

```

    pop(stack);
}
}
}
return (stack->top == -1); // If stack is empty, all brackets are balanced
}

int main() {
    char exp[MAX_SIZE];
    printf("Enter an expression with parentheses: ");
    scanf("%s", exp);
    if (areBracketsBalanced(exp)) {
        printf("Parentheses are balanced\n");
    } else {
        printf("Parentheses are not balanced\n");
    }
    return 0;
}

```

Output:

Enter an expression with parentheses: a+b

Parentheses are balanced

Enter an expression with parentheses: (a+b

Parentheses are not balanced

6(i) IMPLEMENT A QUEUE USING ARRAYS AND LINKED LISTS.

(A) Array-based Queue:

Program:

```
#include <stdio.h>
#include <stdlib.h>
#define MAX_SIZE 100
typedef struct {
    int array[MAX_SIZE];
    int front;
    int rear;
} ArrayQueue;

void initQueue(ArrayQueue *q) {
    q->front = -1;
    q->rear = -1;
}

int isEmpty(ArrayQueue *q) {
    return (q->front == -1 && q->rear == -1);
}

int isFull(ArrayQueue *q) {
    return (q->rear == MAX_SIZE - 1);
}

void enqueue(ArrayQueue *q, int data) {
    if (isFull(q)) {
        printf("Queue is full. Cannot enqueue.\n");
        return;
    }
    if (isEmpty(q)) {
        q->front = 0;
```

```
}
```

```
q->rear++;
```

```
q->array[q->rear] = data;
```

```
}
```

```
int dequeue(ArrayQueue *q) {
```

```
int data;
```

```
if (isEmpty(q)) {
```

```
printf("Queue is empty. Cannot dequeue.\n");
```

```
return -1;
```

```
}
```

```
data = q->array[q->front];
```

```
if (q->front == q->rear) {
```

```
q->front = -1;
```

```
q->rear = -1;
```

```
} else {
```

```
q->front++;
```

```
}
```

```
return data;
```

```
}
```

```
void display(ArrayQueue *q) {
```

```
if (isEmpty(q)) {
```

```
printf("Queue is empty.\n");
```

```
return;
```

```
}
```

```
printf("Queue elements: ");
```

```
for (int i = q->front; i <= q->rear; i++) {
```

```
printf("%d ", q->array[i]);
```

```
}
```

```
printf("\n");
```

```

}
int main() {
    ArrayQueue q;
    initQueue(&q);
    int choice, data;
    while (1) {
        printf("\nQueue Operations:\n");
        printf("1. Enqueue\n");
        printf("2. Dequeue\n");
        printf("3. Display\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("Enter data to enqueue: ");
                scanf("%d", &data);
                enqueue(&q, data);
                break;
            case 2:
                printf("Dequeued element: %d\n", dequeue(&q));
                break;
            case 3:
                display(&q);
                break;
            case 4:
                printf("Exiting program.\n");
                exit(0);
            default:
                printf("Invalid choice. Please try again.\n");

```

```
}  
}  
return 0;  
}
```

Output:

Queue Operations:

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 1

Enter data to enqueue: 3

Queue Operations:

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 1

Enter data to enqueue: 5

Queue Operations:

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 3

Queue elements: 3 5

(B)Linked List-based Queue:

Program:

```
#include <stdio.h>
#include <stdlib.h>
typedef struct Node {
    int data;
    struct Node *next;
} Node;
typedef struct {
    Node *front;
    Node *rear;
} LinkedListQueue;
void initQueue(LinkedListQueue *q) {
    q->front = NULL;
    q->rear = NULL;
}
int isEmpty(LinkedListQueue *q) {
    return (q->front == NULL);
}
Node* createNode(int data) {
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == NULL) {
        printf("Memory allocation failed.\n");
        exit(1);
    }
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
```

```

}
void enqueue(LinkedListQueue *q, int data) {
Node *newNode = createNode(data);
if (isEmpty(q)) {
q->front = newNode;
} else {
q->rear->next = newNode;
}
q->rear = newNode;
}
int dequeue(LinkedListQueue *q) {
int data;
Node *temp;
if (isEmpty(q)) {
printf("Queue is empty. Cannot dequeue.\n");
return -1;
}
temp = q->front;
data = temp->data;
q->front = q->front->next;
free(temp);
return data;
}
void display(LinkedListQueue *q) {
Node *current = q->front;
if (isEmpty(q)) {
printf("Queue is empty.\n");
return;
}
printf("Queue elements: ");

```

```

while (current != NULL) {
printf("%d ", current->data);
current = current->next;
}
printf("\n");
}

int main() {
LinkedListQueue q;
initQueue(&q);
int choice, data;
while (1) {
printf("\nQueue Operations:\n");
printf("1. Enqueue\n");
printf("2. Dequeue\n");
printf("3. Display\n");
printf("4. Exit\n");
printf("Enter your choice: ");
scanf("%d", &choice);
switch (choice) {
case 1:
printf("Enter data to enqueue: ");
scanf("%d", &data);
enqueue(&q, data);
break;
case 2:
printf("Dequeued element: %d\n", dequeue(&q));
break;
case 3:
display(&q);
break;

```

```
case 4:
printf("Exiting program.\n");
exit(0);
default:
printf("Invalid choice. Please try again.\n");
}}
return 0;}
```

Output:

Queue Operations:

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 1

Enter data to enqueue:

Queue Operations:

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 1

Enter data to enqueue: 9

Queue Operations:

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 3

Queue elements: 6 9

**6(ii) DEVELOP A PROGRAM TO SIMULATE A SIMPLE
PRINTER QUEUE SYSTEM.**

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
#define MAX_SIZE 10
typedef struct Node {
    int data;
    struct Node *next;
} Node;
typedef struct {
    Node *front;
    Node *rear;
} Queue;
void initQueue(Queue *q) {
    q->front = NULL;
    q->rear = NULL;
}
int isEmpty(Queue *q) {
    return (q->front == NULL);
}
Node* createNode(int data) {
    Node *newNode = (Node *)malloc(sizeof(Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}
void enqueue(Queue *q, int data) {
    Node *newNode = createNode(data);
```

```

if (isEmpty(q)) {
q->front = newNode;
q->rear = newNode;
} else {
q->rear->next = newNode;
q->rear = newNode;
}
}

int dequeue(Queue *q) {
if (isEmpty(q)) {
printf("Queue is empty. Cannot dequeue.\n");
return -1;
}

int data = q->front->data;
Node *temp = q->front;
q->front = q->front->next;
if (q->front == NULL) {
q->rear = NULL;
}

free(temp);
return data;
}

void display(Queue *q) {
Node *current = q->front;
if (isEmpty(q)) {
printf("Queue is empty.\n");
return;
}

printf("Queue elements: ");
while (current != NULL) {

```

```

printf("%d ", current->data);
current = current->next;
}
printf("\n");
}
int main() {
Queue q;
initQueue(&q);
int choice, data;
while (1) {
printf("\nPrinter Queue System:\n");
printf("1. Add job to queue\n");
printf("2. Print job\n");
printf("3. Display queue\n");
printf("4. Exit\n");
printf("Enter your choice: ");
scanf("%d", &choice);
switch (choice) {
case 1:
printf("Enter job number to add to queue: ");
scanf("%d", &data);
enqueue(&q, data);
break;
case 2:
printf("Printing job number: %d\n", dequeue(&q));
break;
case 3:
display(&q);
break;
case 4:

```

```
printf("Exiting program.\n");
exit(0);
default:
printf("Invalid choice. Please try again.\n");
}
}
return 0;
}
```

Output:

Printer Queue System:

1. Add job to queue
2. Print job
3. Display queue
4. Exit

Enter your choice: 1

Enter job number to add to queue: 45

Printer Queue System:

1. Add job to queue
2. Print job
3. Display queue
4. Exit

Enter your choice: 1

Enter job number to add to queue: 67

Printer Queue System:

1. Add job to queue
2. Print job
3. Display queue
4. Exit

Enter your choice: 3

Queue elements: 45 67

Printer Queue System:

1. Add job to queue
2. Print job
3. Display queue
4. Exit

Enter your choice: 2

Printing job number: 45

Printer Queue System:

1. Add job to queue
2. Print job
3. Display queue
4. Exit

Enter your choice: 3

Queue elements: 67

6(iii) SOLVE PROBLEMS INVOLVING CIRCULAR QUEUES.

Program:

```
#include <stdio.h>
#include <stdlib.h>
#define MAX_SIZE 5
typedef struct {
    int *array;
    int front;
    int rear;
    int capacity;
    int count;
} CircularQueue;

// Function to initialize the circular queue
void initQueue(CircularQueue *q, int capacity) {
    q->array = (int *)malloc(capacity * sizeof(int));
    if (q->array == NULL) {
        printf("Memory allocation failed.\n");
        exit(1);
    }
    q->capacity = capacity;
    q->front = -1;
    q->rear = -1;
    q->count = 0;
}

// Function to check if the circular queue is empty
int isEmpty(CircularQueue *q) {
    return (q->count == 0);
}

// Function to check if the circular queue is full
```

```

int isFull(CircularQueue *q) {
return (q->count == q->capacity);
}

// Function to enqueue an element into the circular queue
void enqueue(CircularQueue *q, int data) {
if (isFull(q)) {
printf("Queue is full. Cannot enqueue.\n");
return;
}
if (isEmpty(q)) {
q->front = 0;
}
q->rear = (q->rear + 1) % q->capacity;
q->array[q->rear] = data;
q->count++;
}

// Function to dequeue an element from the circular queue
int dequeue(CircularQueue *q) {
int data;
if (isEmpty(q)) {
printf("Queue is empty. Cannot dequeue.\n");
return -1;
}
data = q->array[q->front];
if (q->front == q->rear) {
// Queue has only one element, reset front and rear
q->front = -1;
q->rear = -1;
} else {
q->front = (q->front + 1) % q->capacity;
}
return data;
}

```

```

}
q->count--;
return data;
}
// Function to display the elements of the circular queue
void display(CircularQueue *q) {
if (isEmpty(q)) {
printf("Queue is empty.\n");
return;
}
printf("Queue elements: ");
int i = q->front;
while (i != q->rear) {
printf("%d ", q->array[i]);
i = (i + 1) % q->capacity;
}
printf("%d\n", q->array[i]);
}
int main() {
CircularQueue q;
initQueue(&q, MAX_SIZE);
int choice, data;
while (1) {
printf("\nCircular Queue Operations:\n");
printf("1. Enqueue\n");
printf("2. Dequeue\n");
printf("3. Display\n");
printf("4. Exit\n");
printf("Enter your choice: ");
scanf("%d", &choice);

```

```
switch (choice) {
case 1:
printf("Enter data to enqueue: ");
scanf("%d", &data);
enqueue(&q, data);
break;
case 2:
printf("Dequeued element: %d\n", dequeue(&q));
break;
case 3:
display(&q);
break;
case 4:
printf("Exiting program.\n");
exit(0);
default:
printf("Invalid choice. Please try again.\n");
}
}
return 0;
}
```

Output:

Circular Queue Operations:

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter your choice: 1

Enter data to enqueue: 34

Circular Queue Operations:

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter your choice: 1

Enter data to enqueue: 56

Circular Queue Operations:

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter your choice: 1

Enter data to enqueue: 78

Circular Queue Operations:

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter your choice: 3

Queue elements: 34 56 78

**7(i) USE A STACK TO EVALUATE AN INFIX EXPRESSION AND
CONVERT IT TO POSTFIX.**

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#define MAX_SIZE 100
typedef struct {
    char items[MAX_SIZE];
    int top;
} Stack;
// Function prototypes
void initStack(Stack *s);
int isEmpty(Stack *s);
int isFull(Stack *s);
void push(Stack *s, char c);
char pop(Stack *s);
char peek(Stack *s);
int precedence(char op);
void infixToPostfix(char *infix, char *postfix);
// Function to initialize the stack
void initStack(Stack *s) {
    s->top = -1;
}
// Function to check if the stack is empty
int isEmpty(Stack *s) {
    return (s->top == -1);
}
```

```

// Function to check if the stack is full
int isFull(Stack *s) {
return (s->top == MAX_SIZE - 1);
}

// Function to push an item onto the stack
void push(Stack *s, char c) {
if (isFull(s)) {
printf("Stack overflow.\n");
exit(1);
}
s->items[++(s->top)] = c;
}

// Function to pop an item from the stack
char pop(Stack *s) {
if (isEmpty(s)) {
printf("Stack underflow.\n");
exit(1);
}
return s->items[(s->top)--];
}

// Function to return the top element of the stack without popping it
char peek(Stack *s) {
if (isEmpty(s)) {
printf("Stack is empty.\n");
exit(1);
}return s->items[s->top];
}

// Function to determine precedence of operators
int precedence(char op) {

```

```

switch (op) {
case '+':
case '-':
return 1;
case '*':
case '/':
return 2;
case '^':
return 3;
default:
return -1;
}}

// Function to convert infix expression to postfix
void infixToPostfix(char *infix, char *postfix) {
Stack stack;
initStack(&stack);
int i, j;
char c;
for (i = 0, j = 0; infix[i] != '\0'; i++) {
c = infix[i];
if (isalnum(c)) {
postfix[j++] = c;
} else if (c == '(') {
push(&stack, c);
} else if (c == ')') {
while (!isEmpty(&stack) && peek(&stack) != '(') {
postfix[j++] = pop(&stack);
}
if (!isEmpty(&stack) && peek(&stack) != '(') {
printf("Invalid expression.\n");
}
}
}

```

```

exit(1);
} else {
pop(&stack); // Discard the '('
}
} else {
while (!isEmpty(&stack) && precedence(c) <= precedence(peek(&stack))) {
postfix[j++] = pop(&stack);
}
push(&stack, c);
}}
// Pop any remaining operators from the stack
while (!isEmpty(&stack)) {
postfix[j++] = pop(&stack);
}
postfix[j] = '\0'; // Null-terminate the postfix expression
}

int main() {
char infix[MAX_SIZE], postfix[MAX_SIZE];
printf("Enter infix expression: ");
fgets(infix, MAX_SIZE, stdin);
infix[strcspn(infix, "\n")] = '\0'; // Remove trailing newline if present
infixToPostfix(infix, postfix);
printf("Postfix expression: %s\n", postfix);
return 0;
}

```

Output:

Enter infix expression: a+b=c

Postfix expression: ab+c=

**7(ii) CREATE A PROGRAM TO DETERMINE WHETHER A
GIVEN STRING IS A PALINDROME OR NOT.**

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#define MAX_SIZE 100
typedef struct {
    char items[MAX_SIZE];
    int top;
} Stack;
typedef struct {
    char items[MAX_SIZE];
    int front, rear;
} Queue;
void initStack(Stack *s) {
    s->top = -1;
}
int isEmptyStack(Stack *s) {
    return (s->top == -1);
}
int isFullStack(Stack *s) {
    return (s->top == MAX_SIZE - 1);
}
void push(Stack *s, char c) {
    if (!isFullStack(s))
        s->items[++(s->top)] = c;
}
```

```

char pop(Stack *s) {
if (!isEmptyStack(s))
return s->items[(s->top)--];
}

void initQueue(Queue *q) {
q->front = 0;
q->rear = -1;
}

int isEmptyQueue(Queue *q) {
return (q->rear < q->front);
}

int isFullQueue(Queue *q) {
return (q->rear == MAX_SIZE - 1);
}

void enqueue(Queue *q, char c) {
if (!isFullQueue(q))
q->items[++(q->rear)] = c;
}

char dequeue(Queue *q) {
if (!isEmptyQueue(q))
return q->items[(q->front)++];
}

int isPalindrome(char *str) {
int len = strlen(str);
Stack stack;
Queue queue;
initStack(&stack);
initQueue(&queue);
for (int i = 0; i < len; i++) {
char c = tolower(str[i]);

```

```

if (isalpha(c)) {
    push(&stack, c);
    enqueue(&queue, c);
}
}
while (!isEmptyStack(&stack) && !isEmptyQueue(&queue)) {
    char stackTop = pop(&stack);
    char queueFront = dequeue(&queue);
    if (stackTop != queueFront)
        return 0;
}
return 1;
}

int main() {
    char str[MAX_SIZE];
    printf("Enter a string: ");
    fgets(str, MAX_SIZE, stdin);
    str[strcspn(str, "\n")] = '\0';
    if (isPalindrome(str))
        printf("The string \"%s\" is a palindrome.\n", str);
    else
        printf("The string \"%s\" is not a palindrome.\n", str);
    return 0;
}

```

Output:

Enter a string: radar

The string "radar" is a palindrome.

**7(iii)IMPLEMENT A STACK OR QUEUE TO PERFORM
COMPARISON AND CHECK FOR SYMMETRY.**

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAX_SIZE 100
typedef struct {
    char items[MAX_SIZE];
    int top;
} Stack;
void initStack(Stack *s) {
    s->top = -1;
}
int isEmptyStack(Stack *s) {
    return (s->top == -1);
}
int isFullStack(Stack *s) {
    return (s->top == MAX_SIZE - 1);
}
void push(Stack *s, char c) {
    if (!isFullStack(s))
        s->items[++(s->top)] = c;
}
char pop(Stack *s) {
    if (!isEmptyStack(s))
        return s->items[(s->top)--];
}
int isSymmetric(char *str) {
```

```

int len = strlen(str);
Stack stack;
initStack(&stack);
for (int i = 0; i < len; i++) {
    if (str[i] == '(' || str[i] == '[' || str[i] == '{') {
        push(&stack, str[i]);
    } else if (str[i] == ')' || str[i] == ']' || str[i] == '}') {
        if (isEmptyStack(&stack))
            return 0;
        char c = pop(&stack);
        if ((str[i] == ')' && c != '(') ||
            (str[i] == ']' && c != '[') ||
            (str[i] == '}' && c != '{')) {
            return 0;
        }
    }
}
return isEmptyStack(&stack);
}

int main() {
    char str[MAX_SIZE];
    printf("Enter a string: ");
    fgets(str, MAX_SIZE, stdin);
    str[strcspn(str, "\n")] = '\0';
    if (isSymmetric(str))
        printf("The string \"%s\" is symmetric.\n", str);
    else
        printf("The string \"%s\" is not symmetric.\n", str);
    return 0;
}

```

Output:

Enter a string: "{[()]}"

The string "{[()]}" is symmetric.

Enter a string: "{[()]}("

The string "{[()]}(" is not symmetric.

8(i) IMPLEMENT A HASH TABLE WITH COLLISION RESOLUTION TECHNIQUES.

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define TABLE_SIZE 10
// Define structure for key-value pair
typedef struct {
    char *key;
    int value;
} KeyValuePair;
// Define structure for a node in the linked list (for chaining)
typedef struct Node {
    KeyValuePair pair;
    struct Node *next;
} Node;
// Define structure for the hash table
typedef struct {
    Node *buckets[TABLE_SIZE];
} HashTable;
// Hash function
int hash(char *key) {
    int hash_val = 0;
    while (*key != '\0') {
        hash_val += *key;
        key++;
    }
    return hash_val % TABLE_SIZE;
```

```

}

// Initialize the hash table
void initHashTable(HashTable *ht) {
    for (int i = 0; i < TABLE_SIZE; i++) {
        ht->buckets[i] = NULL;
    }
}

// Insert a key-value pair into the hash table
void insert(HashTable *ht, char *key, int value) {
    int index = hash(key);
    // Create a new node for the key-value pair
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == NULL) {
        fprintf(stderr, "Memory allocation failed\n");
        exit(1);
    }
    newNode->pair.key = strdup(key); // Duplicate the key string
    newNode->pair.value = value;
    newNode->next = NULL;
    // Insert the new node into the bucket (chaining)
    if (ht->buckets[index] == NULL) {
        ht->buckets[index] = newNode;
    } else {
        Node *current = ht->buckets[index];
        while (current->next != NULL) {
            current = current->next;
        }
        current->next = newNode;
    }
}

```

```

// Search for a key in the hash table and return its value
int search(HashTable *ht, char *key) {
    int index = hash(key);
    Node *current = ht->buckets[index];
    while (current != NULL) {
        if (strcmp(current->pair.key, key) == 0) {
            return current->pair.value;
        }
        current = current->next;
    }
    return -1; // Key not found
}

// Main function
int main() {
    HashTable ht;
    initHashTable(&ht);
    char key[100];
    int value;
    // Input key-value pairs from the user
    printf("Enter key-value pairs (type 'exit' to stop):\n");
    while (1) {
        printf("Key: ");
        scanf("%s", key);
        if (strcmp(key, "exit") == 0) break;
        printf("Value: ");
        scanf("%d", &value);
        insert(&ht, key, value);
    }
    // Search for keys entered by the user

```

```
printf("\nEnter keys to search (type 'exit' to stop):\n");

while (1) {
printf("Key: ");
scanf("%s", key);
if (strcmp(key, "exit") == 0) break;
int result = search(&ht, key);
if (result != -1)
printf("Value for key '%s': %d\n", key, result);
else
printf("Key '%s' not found\n", key);
}
return 0;
}
```

Output:

```
Enter key-value pairs (type 'exit' to stop):
Key: 1
Value: 145
Key: 2
Value: 678
Key: exit
Enter keys to search (type 'exit' to stop):
Key: 1
Value for key '1': 145
Key: 2
Value for key '2': 678
```

**8(ii) WRITE A PROGRAM TO IMPLEMENT A SIMPLE CACHE
USING HASHING.**

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define CACHE_SIZE 10
// Define structure for cache node
typedef struct CacheNode {
    char *key;
    char *value;
    struct CacheNode *next;
} CacheNode;
// Define structure for cache
typedef struct {
    CacheNode *hash[CACHE_SIZE];
} Cache;
// Function to create a new cache node
CacheNode *createCacheNode(char *key, char *value) {
    CacheNode *node = malloc(sizeof(CacheNode));
    if (!node) {
        fprintf(stderr, "Memory allocation failed\n");
        exit(1);
    }
    node->key = strdup(key);
    node->value = strdup(value);
    node->next = NULL;
    return node;
}
```

```

}

// Function to calculate hash value
int hash(char *key) {
    int hash_val = 0;
    while (*key) {
        hash_val += *key++;
    }
    return hash_val % CACHE_SIZE;
}

// Function to insert a key-value pair into the cache
void put(Cache *cache, char *key, char *value) {
    int index = hash(key);
    CacheNode *node = createCacheNode(key, value);
    node->next = cache->hash[index];
    cache->hash[index] = node;
}

// Function to get the value associated with a key from the cache
char *get(Cache *cache, char *key) {
    int index = hash(key);
    CacheNode *node = cache->hash[index];
    while (node != NULL) {
        if (strcmp(node->key, key) == 0) {
            return node->value;
        }
        node = node->next;
    }
    return NULL; // Key not found
}

// Main function
int main() {

```

```

Cache cache;
for (int i = 0; i < CACHE_SIZE; i++) {
    cache.hash[i] = NULL;
}
char key[50], value[50];
int choice;
do {
    printf("\n1. Insert a key-value pair\n");
    printf("2. Retrieve value for a key\n");
    printf("3. Exit\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);
    switch (choice) {
        case 1:
            printf("Enter key: ");
            scanf("%s", key);
            printf("Enter value: ");
            scanf("%s", value);
            put(&cache, key, value);
            printf("Key-value pair inserted into cache.\n");
            break;
        case 2:
            printf("Enter key to retrieve value: ");
            scanf("%s", key);
            char *result = get(&cache, key);
            if (result)
                printf("Value for key '%s': %s\n", key, result);
            else
                printf("Key '%s' not found\n", key);
            break;
    }
} while (choice != 3);

```

```

case 3:
printf("Exiting...\n");
break;
default:
printf("Invalid choice. Please try again.\n");
}
} while (choice != 3);
// Free allocated memory
for (int i = 0; i < CACHE_SIZE; i++) {
CacheNode *current = cache.hash[i];
while (current != NULL) {
CacheNode *temp = current;
current = current->next;
free(temp->key);
free(temp->value);
free(temp);
}
}
return 0;
}

```

Output:

```

1. Insert a key-value pair
2. Retrieve value for a key
3. Exit
Enter your choice: 1
Enter key: red
Enter value: apple
Key-value pair inserted into cache.
1. Insert a key-value pair

```

2. Retrieve value for a key

3. Exit

Enter your choice: 1

Enter key: yellow

Enter value: mango

Key-value pair inserted into cache.

1. Insert a key-value pair

2. Retrieve value for a key

3. Exit

Enter your choice: 2

Enter key to retrieve value: yellow

Value for key 'yellow': mango